

Load-Bearing but Redundant: Rethinking Physics Injection in Shared-Latent World Models

Anonymous submission

Abstract

A world model predicts how an environment evolves in response to actions by rolling an internal representation forward, and is expected to respect physical regularities such as how objects move and collide. We study JEPA-style latent world models and find a puzzle: trained with no physics supervision, such a model already recovers object position well from its representation, predicts one step almost exactly, and—as counterfactual intervention confirms—actually uses that state when rolling forward, yet its multi-step rollouts still drift away from the very dynamics it encodes. The popular fix is to inject physical structure into the latent representation. We test it systematically within the family it names—nine variants that write physics onto the coordinates of the shared latent, spanning hard and soft constraints on both physical quantities and physical laws, with labeled and label-free supervision—across three controlled physics domains and two realistic simulation benchmarks, and find injection fails in 26 of 27 cases; even supplying the *correct* physical constraint does not help. One account fits every test we ran: the physical state is already encoded, redundantly, across a *shared latent*, so an injected copy carries no new signal while the constraint still competes for capacity and gradient. Intervening on a trained model shows the injected slot is *load-bearing but redundant*: perturbing it moves the model’s own black-box state $1.5\text{--}2.0\times$, and swapping it for another trajectory’s makes prediction follow that trajectory 47–72% of the way—yet the black box keeps its own copy, and prediction error does not move. Both routes carry load; the second carries nothing new. What the model lacks is not the object’s physical state but the ability to evolve that state over long horizons: fixing the training procedure alone, adding no physics, cuts rollout error $2.2\text{--}8.3\times$ on identical code and metrics. To help a latent world model, injected structure should supply what the representation is missing—*how the state evolves, not what the state is*.

1 Introduction

A *world model* rolls an internal representation forward to predict how an environment evolves under an agent’s actions—a simulator to plan with instead of acting in the world (Ha and Schmidhuber 2018; LeCun 2022); for a physical environment, it should respect how objects move, fall, and collide. JEPA-style latent world models (Maes 2025; Balestriero and LeCun 2025) present a puzzle here. Trained on PhyWorld (Kang et al. 2025) with no physics supervision,

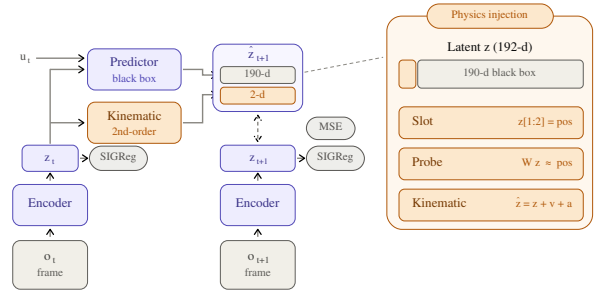


Figure 1: Physics injection into the LeWM shared-latent world model (*amber* = what injection adds). The encoder maps each frame to a 192-d latent; a black-box transformer rolls it forward given the action. Physics enters a 2-d position *slot* carved from the 192 dimensions (right): a hard pin of $z[1:2]$ to position, a supervised *probe*, or a *kinematic* head evolving the slot by a second-order rule—while the other 190 “black-box” dimensions stay free. The 2-vs-190 split is the crux: the black box carries its own copy of the same state, so whatever is injected into the slot is a *duplicate*.

such a model recovers object position by linear probing (ρ up to 0.96) and predicts one step almost exactly (cosine 0.98–0.99)—yet rolled out autoregressively, its predictions drift away from the very dynamics it encodes (collision cosine 0.24 by horizon 28), and out-of-distribution (OOD) physics multiplies rollout error several-fold. The state is present; the model fails to *evolve* it.

The popular fix is to *inject physical structure into the latent* (Figure 1): a dedicated slot pinned to the physical state, a supervised probe that reads it out, or a hard-wired dynamics rule. Injection has real successes in adjacent model classes—architectures routing prediction through a low-dimensional physical state by construction (Mao, Uma-sudhan, and Ruchkin 2024; Peper et al. 2025), deep supervision where the supervised quantities occupy a large fraction of a compact latent (Zahorodnii 2025), a decomposed JEPA latent on numerical time series (Nie et al. 2026)—and scaling alone does not substitute: video generators trained

at scale still fail OOD physics (Kang et al. 2025). But for a *shared, high-dimensional* latent, injection has only been tried in isolated, incomparable settings, never against the confound that dominates the outcome: the training protocol.

We answer this with a systematic, controlled anatomy on a JEPA world model. We cross nine injection variants—spanning hard and soft constraints, targeting the physical *state* (what it is) or its *evolution* (how it changes), under labeled and label-free supervision—with two training regimes and three controlled physics domains, then corroborate on two realistic-simulation benchmarks (Bear et al. 2021; Tung et al. 2023).

Across the full grid, the structure does not help: injection degrades or fails to improve prediction in 26 of 27 mechanism–domain cells. Supplying the *correct* evolution rule—the gravitational law $a=g$ wired into the state update—does no better than a freely learned acceleration where $a=g$ is the ground truth (0.151 vs. 0.149 nMSE on parabola). Co-training from scratch does not rescue it either: injection stays harmful or at parity in every cell, never helping—closing the defense that physics must be baked in during pretraining. The one cell that does improve reverses sign on the other two domains: using it would mean knowing each domain’s dynamics in advance.

Why should the *correct* physics hurt? Because the physical state is *already there*. A pinned slot occupies 2 of the 192 latent dimensions, but the other 190 redundantly encode the same position—decoding it nearly as accurately as the full latent—so an injected copy adds no information the representation lacks, while the constraint still competes with prediction for capacity and gradient. Redundancy accounts for the absent gain, competition for the harm. What injection adds is not a route the prediction lacked but a *second* route to a state it already had, and intervening on a trained model shows prediction takes both: steering the slot moves the model’s own black-box state by 1.5–2.0 $\times$ while a norm-matched random direction moves it ≈ 0 , and swapping the slot for another trajectory’s makes prediction follow that trajectory 47–72% of the way where a model without a slot follows 1.5–11%. The injected slot is thus *load-bearing but redundant*—used, and carrying nothing the black box was not already carrying. This also explains why strengthening it cannot help: if the slot’s $\sim 1\%$ share of the prediction loss were what starves it, up-weighting should remove the harm, and it does not—re-weighting to saturation recovers only *parity*, never a net gain, because the constraint was never short of gradient, only short of new information.

What the representation lacks is not the state but the ability to *evolve* it over long horizons, and that gap is fixable with no physics at all: free rollout, with the horizon matched to each domain’s dynamics, cuts rollout error 2.2–8.3 $\times$ on the identical code, metrics, and seeds that judged every injection cell—pinning the failure on the injected structure rather than the pipeline or the yardstick. The lesson is constructive: an inductive bias must supply what a shared latent is missing (how the state evolves), not re-encode what it already holds. If that reading is right, only an architecture can remove the redundancy; no loss weight can. Porting one that makes the physical state the *sole* carrier (Mao, Umasudhan,

and Ruchkin 2024) does remove it, but it collapses once an OOD shift reaches its encoder (ρ 0.33 vs. 0.89): removing the redundancy is not, by itself, a fix. Every finding above replicates on a second, frozen-DINOv2 backbone.

This paper contributes:

- to our knowledge, the first systematic anatomy of physics injection in *shared-latent* visual world models—nine variants $\times$ two training regimes $\times$ three domains, plus two realistic-simulation benchmarks—finding injection no better than baseline in 26 of 27 cells, and replicating on a frozen-DINOv2 backbone (§4.2, §4.5);
- a mechanistic account, *load-bearing but redundant*, established by intervention rather than inferred: steering, counterfactual patching, and per-dimension Jacobians agree the injected slot *is* used, while the 190 black-box dimensions keep an undiminished copy of the same position—so prediction reads two copies of one state, and re-weighting to saturation buys only parity. An extrinsic port that makes the state the sole carrier helps in-distribution but collapses under encoder-ODD shift (§4.3, §4.5);
- a controlled factorial showing the dominant variable is the training protocol, not physical structure: free rollout alone cuts rollout error 2.2–8.3 $\times$ with no physics added, doubling as the positive control against implementation and metric artifacts (§4.4).

2 Related Work

Physically structured world models. A growing line argues that world models should carry explicit physical state: physically interpretable models route prediction through a dedicated low-dimensional state (Mao, Umasudhan, and Ruchkin 2024; Peper et al. 2025); deep supervision adds physical readouts to the loss and helps in a low-dimensional state latent, where the supervised quantities occupy $\sim 38\%$ of the representation (Zahorodnii 2025); closest to ours, Phys-JEPA supervises a decomposed JEPA latent and reports gains (Nie et al. 2026)—but on numerical time series, not the visual latents we study. Physics-informed losses (Raissi, Perdikaris, and Karniadakis 2019), conservation-structured dynamics (Greydanus, Dzamba, and Yosinski 2019; Cranmer et al. 2020), graph simulators over *explicit* particle state (Battaglia et al. 2016; Sanchez-Gonzalez et al. 2020), and restricted-attention transformers on numerical trajectories (Liu et al. 2026) share a pattern: structure works when the physical state *is* the representation, not a small part of a shared one. We do not dispute these results in their original settings; we test the transferable core of these proposals inside a shared-latent *visual* JEPA model and find all of them unhelpful or harmful (§4.2); our extrinsic port (§4.5) attributes the gap to architecture, not to the physics.

Latent world models and physics benchmarks. JEPA-style models predict in representation space (Assran et al. 2023; Bardes et al. 2024; Balestrierio and LeCun 2025; Maes 2025); object-level masking structures the JEPA objective rather than the latent’s physical content (Nam et al. 2026); DINO-WM plans over frozen features (Zhou et al. 2025;

Oquab et al. 2024)—our cross-backbone control; Dreamer-line models train on imagined rollouts (Ha and Schmidhuber 2018; Hafner et al. 2025). PhyWorld (Kang et al. 2025) supplies controlled physics videos with explicit ID ranges, showing scaled video generators memorize cases rather than laws; Physion/Physion++ (Bear et al. 2021; Tung et al. 2023) add realistic—not camera-captured—simulation. We apply PhyWorld’s OOD protocol to *latent* models rather than pixel-space generators.

Rollout training and exposure bias. Training predictors on their own outputs is a classic remedy for compounding error (Bengio et al. 2015; Venkatraman, Hebert, and Bagnell 2015; Goyal et al. 2016). We claim no novelty for free rollout; its role here is methodological (§3.4): the confound prior injection studies never controlled, and a positive control separating “structure does not help” from “the implementation or metrics are broken.”

3 Method: A Design Space of Physical Injection

We answer the question of §1 by construction: design the ways physics can be injected into a shared latent, state what each design is supposed to achieve, and lay out the experiments that test the mechanism behind it.

3.1 Setting and Notation

We study LeWM (Maes 2025), a compact JEPA-style action-conditioned world model (~ 10 M parameters): an encoder E_θ (ViT-tiny) maps frame o_t to a latent $z_t \in \mathbb{R}^D$ with $D=192$, and a causal-transformer predictor F_ϕ rolls latents forward from the history and action u_t , i.e. $\hat{z}_{t+1} = F_\phi(z_{\leq t}, u_t)$. Training minimizes a prediction loss, an anti-collapse regularizer (Balestriero and LeCun 2025), and—when physics is injected—an auxiliary physical loss:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \beta \text{SIGReg}(z) + \lambda \mathcal{L}_{\text{phys}}. \quad (1)$$

Here $\mathcal{L}_{\text{pred}} = \frac{1}{D} \sum_i w_i (\hat{z}_{t+1,i} - z_{t+1,i})^2$ with $w_i=1$ by default; *re-weighting* raises w_i to w on the 2 slot dimensions (w swept $1 \rightarrow 300$), while λ scales the soft auxiliary losses—two distinct dials. *Teacher forcing* trains F_ϕ on one-step transitions from encoded ground-truth context; *free rollout* feeds predictions back for H autoregressive steps ($H=8$ unless stated) and supervises the whole rollout. The kinematic head (Group 2 below) replaces the slot’s transition by a second-order update

$$\hat{z}_{t+1}^{[0:2]} = z_t^{[0:2]} + v_t \Delta t + \frac{1}{2} a \Delta t^2, \quad (2)$$

where v_t is the finite-difference slot velocity and the acceleration a is either a learned MLP(z_t, u_t) or a single learnable constant—the strict $a=g$ head. Unless stated otherwise the encoder is initialized from the public PushT checkpoint; §4.2 additionally studies from-scratch training.

3.2 Injection Design Space: Three Groups, Nine Variants

We study one family: *intrinsic coordinate injection*—physics written onto the coordinates of a latent the predictor already shares, leaving the encoder E_θ and predictor F_ϕ

otherwise untouched. This is the family targeted by the familiar “just add a physics term” prescription; within it we vary three orthogonal axes: injecting *what the motion state is* vs. *how it evolves*, under *hard* vs. *soft* constraints, with *labeled* vs. *label-free* supervision (Table 1).

Two families lie outside it and our results do not speak to them: designs that change what the representation *is* (object-centric slots, relational graphs; Locatello et al. 2020; Battaglia et al. 2016; Sanchez-Gonzalez et al. 2020) and designs that replace the predictor’s functional form—the *transition operator* (Hamiltonian/symplectic integrators, learned contact; Cranmer et al. 2020). Both alter the architecture rather than annotate its coordinates—where §4.3 argues the leverage lies; we test one as an external control (§3.5).

Orthogonally, we cross injection with the *training regime*—fine-tuning a pretrained encoder vs. from-scratch co-training, injection on/off, a 2×2 per domain. This targets the objection that a constraint grafted onto a settled representation never gets to shape it: co-training from random initialization puts the constraint in from the first gradient step, so for the objection to hold, injection would have to help—or at least stop hurting—in that regime (§4.2).

3.3 Mechanistic Hypothesis and Its Tests

Two properties are easy to conflate, and separating them is what the tests below are for. A channel is **load-bearing** if perturbing it changes the prediction—causal, measurable only by intervention. A channel is **marginal** if it supplies information the rest of the latent does not—informational, visible in whether adding it improves prediction. Probing establishes neither (Elazar et al. 2021); the two come apart exactly when a representation is redundant, which is our case.

Hypothesis (the redundancy problem). The physical slot occupies 2 of 192 dimensions, while the black-box channel *redundantly* encodes the same position—so an injected slot duplicates what the latent already carries. Prediction may well read the injected copy (§4.3 shows it does), but reading a duplicate cannot supply information the representation lacks: injection adds no new signal while still consuming capacity and gradient share. We test this account in four ways, each admitting a specific outcome that would falsify it:

1. **Is the physical constraint injected?** Split the latent into the 2 slot dimensions and the 190 black-box dimensions and decode position from each separately, with a random-2-dimension control (Hewitt and Liang 2019). If the black box alone cannot decode position, the redundancy claim is false.
2. **Is the constraint acting at all?** Measure the weighted physical loss relative to the prediction loss (a gradient-magnitude proxy) and the encoder’s participation ratio. If neither moves, the failure is a bug; if the term dominates and dimensionality collapses, the constraint is acting—and competing with prediction.
3. **Does re-weighting help?** Sweep the slot weight w from 1 to 300 (Eq. 1). If harm does not respond to the weight, the gradient-share mechanism is wrong; if it responds but saturation still cannot rescue every domain, insufficient physical loss is not the root cause.

Variant	What it does, and what it tests
<i>Group 1: inject the state</i> — what the motion is (position, velocity) slot (structpos)	
	<i>hard.</i> pin dims [0:2] to the true position (the object’s x, y coordinates; a fixed index range, not an attention slot (Locatello et al. 2020)): the most direct state injection
+reweight	<i>hard.</i> raise the slot’s share of the prediction loss ($w=30$, Eq. 1): if failure is due to the slot’s $\sim 1\%$ gradient share, this should rescue it
+velocity	<i>hard.</i> encode velocity too: tests whether it matters <i>which</i> quantity is injected, not just that something is
probe	<i>soft.</i> linear head reads position from the latent: replicates the deep-supervision recipe (Zahorodnii 2025) in a high-dimensional visual latent
+slot	<i>both.</i> soft + hard combination: tests complementarity
<i>Group 2: inject the evolution</i> — how it <i>changes</i> (a dynamics rule) free MLP	
	<i>hard.</i> attach $z+v+a$ update with learned a : upgrade from pinning state to pinning evolution
strict $a=g$	<i>hard.</i> exact gravitational form, one learnable constant: closes the “your physics was wrong” objection. Doubles as Group 3’s labeled control (grounded prior)—the two specifications coincide
consistency	<i>soft.</i> finite-difference velocity of the rolled-out positions must match ground truth; assumes no form for a , designed for collision impulses
<i>Group 3: drop the labels</i> — is ground truth needed?	
label-free	<i>hard.</i> the kinematic head is attached but nothing pins the slot to ground truth: the only injection available for raw video. Its labeled control is exactly strict $a=g$ above

Table 1: Nine injection variants in three groups, following the **target** axis (state / evolution / label-free prior). Per variant, *hard* forces a latent dimension to equal the physical quantity; *soft* adds an auxiliary loss that only encourages it. Adding labels to the label-free prior yields exactly the strict $a=g$ specification, so that variant serves two roles—the correct-physics test and the labeled control—and appears once. All variants use the strongest training protocol (§3.4).

4. **Is the injected slot load-bearing?** Tests 1–3 never contain the variable that matters here—which channel prediction reads. We therefore intervene on trained models three ways: *steering* (offset the position one channel decodes, read the change off the *other*, against a norm-matched random direction), *counterfactual patching* (replace a channel with a donor trajectory’s values, measure how far the rollout follows the donor, against a model with no slot to replace), and *per-dimension Jacobians* (differentiate predicted position w.r.t. each latent coordinate—no intervention designed). If the slot bears no load, all three read zero on it.

3.4 The Training-Protocol Control: Free Rollout

Free rollout serves two methodological roles; neither is a novelty claim (it is scheduled sampling, Bengio et al. 2015).

The training protocol as a controlled factor. Physical structure has never been compared against the training protocol on the same baseline; prior studies treat the protocol as background. We set the protocol first—replacing teacher forcing with free rollout—then ask whether structure adds incremental value. Teacher forcing hides compounding error at training time (*exposure bias*); free rollout exposes it, injecting no physics. But the gain is more than robustness: supervising an H -step rollout grades the predictor on *evolving* the state over the horizon rather than reproducing one-step transitions—an evolution-targeted training signal needing no physics labels, training *how the state evolves* without touching *what the state is*. The horizon H must match dynamics complexity: domains with late causal events need horizons long enough to contain them.

Positive control. A negative result invites two deflations—“your implementation is broken” or “your metrics cannot detect improvement.” A protocol change producing large gains on identical code and metrics rules out both, pinning the failure on the injected structure itself.

3.5 External and Cross-Backbone Controls

Two controls probe our account from outside the LeWM backbone: one tests whether an architecture that *removes* the redundancy succeeds where injection fails, the other whether the conclusions survive a different encoder.

Extrinsic port. We faithfully port the physically interpretable world model (PIWM) of Mao, Umasudhan, and Ruchkin (2024)—VAE encoder, supervised extractor to a low-dimensional physical state, known-equation dynamics—onto the same domains and OOD protocol. Extrinsic designs make the physical state the *sole* carrier, removing the redundancy by construction; the port tests that prediction from the outside.

Cross-backbone control. We replicate the core protocol on a second JEPA-family instantiation in the style of DINO-WM (Zhou et al. 2025): a *frozen* DINOv2-small encoder (Oquab et al. 2024) that has never seen PhyWorld, the trainable projector as adapter, and the identical predictor stack, losses, seeds, and evaluation—differing from LeWM in backbone provenance, dimensionality, and in whether the injected losses can reshape the encoder at all.

4 Experiments

4.1 Setup

Model. The compact backbone of §3.1 allows the complete factorial anatomy (~ 200 training runs). Every primary result—the 27-cell scan and its baselines, the re-weighting sweep, the training-regime 2×2 , and the Physion++ injection table—is three-seed (3072/1234/42).

Datasets and OOD protocol. All benchmarks are *visual* world-modeling tasks (RGB video), not the numerical/tabular sequences of prior physics-injection work (§2),

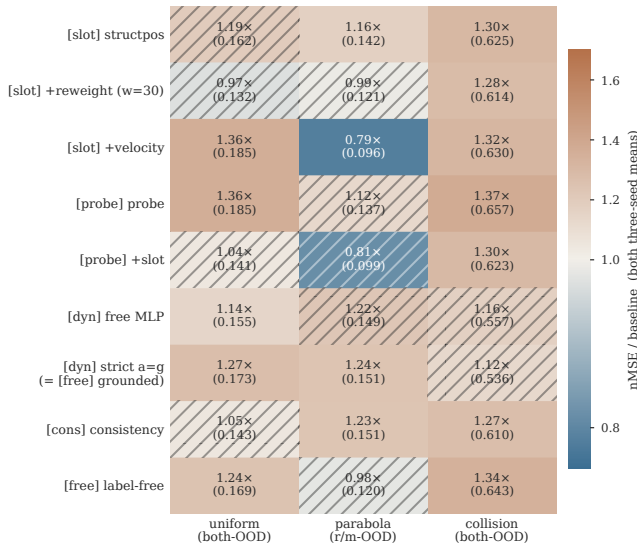


Figure 2: Every injection variant $\times$ domain vs. the free-rollout baseline (nMSE ratio; every cell and the baseline are three-seed means). Warm = worse, blue = better; *hatched* = within noise (the cell’s seeds overlap the baseline’s), solid = separated in every seed. Injection is at or below baseline in 26 of 27 cells (15 worse, 11 within noise); the sole seed-robust gain is velocity+slot on parabola (0.79 $\times$). Strict $a=g$ and the grounded prior are one configuration launched twice; the merged row pools all four draws.

so injected state is always a small fraction of a high-dimensional visual latent. *PhyWorld* (Kang et al. 2025) provides three single-mechanism domains—*uniform motion*, *parabola*, *collision* (zero/constant/impulsive acceleration)—each with 1,000 ID trajectories, evaluated on four partitions (ID, r/m-OOD, v-OOD, both-OOD) whose single-variable OOD ranges isolate physical extrapolation. Two *realistic-simulation* benchmarks (rendered, not camera video) add realism: *Physion* (Bear et al. 2021) for zero-shot object-contact prediction, and *Physion++* (Tung et al. 2023) (ten physical-property scenarios) for direct training with rollout to horizon 64.

Metrics and decision rules. Verdicts use two scale-aware, *external* metrics—latent nMSE ($\|\hat{\mathbf{z}} - \mathbf{z}\|^2$ over target variance) and pixel PSNR of decoded rollouts—external because no injection variant optimizes them directly, and they agree in every case we examined. Latent cosine and probe decodability ρ (Alain and Bengio 2016) are *diagnostics only*: both are duals of the auxiliary losses and rise mechanically when those are optimized. nMSE fails oppositely—its denominator degenerates when target variance collapses—so parabola is judged on its clean r/m-OOD partition (pathology catalogue: Appendix F). Splits are trajectory-level; three-seed results report mean $\pm$ sample std.

4.2 Injection Fails in 26 of 27 Cells

Injection degrades or fails to improve prediction in 26 of 27 mechanism-domain cells (15 worse in every seed, 11

Domain	Regime	phys. off	phys. on	Δ
uniform	scratch	0.222 $\pm$ 0.028	0.657 $\pm$ 0.095	+0.435
	fine-tune	0.136	0.173	+0.037
parabola (r/m-OOD)	scratch	0.476 $\pm$ 0.169	0.447 $\pm$ 0.063	−0.029
	fine-tune	0.122	0.151	+0.029
collision	scratch	0.502 $\pm$ 0.043	0.648 $\pm$ 0.013	+0.147
	fine-tune	0.479	0.536 [°]	+0.057

Table 2: Injection on/off $\times$ training regime (latent nMSE $\downarrow$, hardest clean split; injected structure: strict $a=g$ head + fixed slot). Both regimes are three-seed; [°] = within seed noise. Δ = cost of switching injection on. No cell shows injection helping.

within noise; Figure 2, per-seed values in Appendix B). Read group by group, the scan answers every design rationale of Table 1. *Hard state injection*: the slot faithfully absorbs position (supervision loss 2.53 $\rightarrow$ 0.015; slot decodability 0.31 $\rightarrow$ 0.96), yet prediction never improves (1.16–1.30 $\times$ worse or within noise): the constraint acts, and buys nothing. *Soft injection*: the deep-supervision recipe, effective where supervised quantities occupy $\sim$ 38% of a compact latent (Zahorodnii 2025), does not transfer to a 192-D visual latent where they occupy 2% (1.36–1.37 $\times$ on the both-OOD domains), and combining soft with hard does not rescue. *Correct physics does not help*: the strict $a=g$ head worsens uniform by 1.27 $\times$ and matches a freely learned acceleration on parabola (0.151 vs. 0.149). *The purpose-built design does not help its target*: the consistency loss, designed for collision impulses, still degrades collision (1.27 $\times$). *Labels are not the issue*: the label-free prior and its labeled control—the $a=g$ row, the identical specification with ground truth added—fail alike (collision 1.34 $\times$ vs. 1.12 $\times$): the failure is architectural, not informational.

The sole exception is the mechanism’s signature, not a usable prior: adding *velocity* to the re-weighted slot helps on parabola (0.122 $\pm$ 0.007 $\rightarrow$ 0.096 $\pm$ 0.007, every seed lower) because there velocity is the extrapolable driving variable; the same encoding is harmful on uniform (velocity is a redundant constant) and collision (velocity jumps discontinuously). A prior that requires knowing each domain’s dynamics in advance, and reverses sign across domains, is a diagnostic, not a method—and its best case is an order of magnitude below the protocol lever of §4.4.

From-scratch co-training does not rescue injection.

The remaining defense—structure must be baked in during pretraining—does not hold (Table 2). Over three seeds, co-training from random initialization leaves injection harmful on uniform (+0.435, Welch 95% CI [+0.276, +0.594]) and collision (+0.147, [+0.074, +0.220]), and at parity on parabola (−0.029, [−0.318, +0.260]); its from-scratch baseline is too seed-unstable to resolve an effect this small). Parity is not a rescue—the defense requires injection to *help*, and in no cell does it; baking the structure in from the start even produces the single worst failure in the grid (+0.435 on uniform, vs. +0.037 fine-tuned).

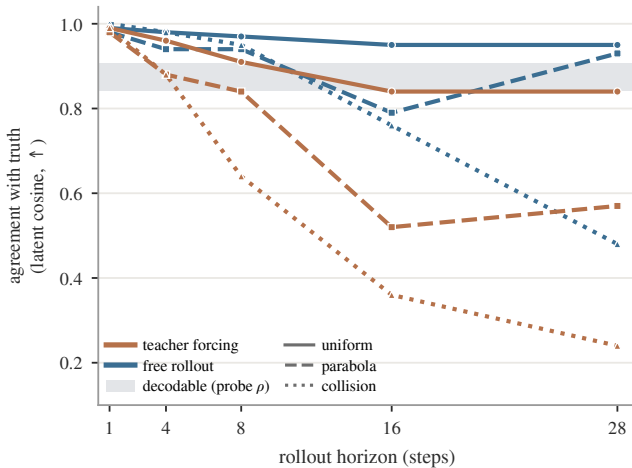


Figure 3: Decodable, used, and still drifting. The shaded band is position decodability from frame embeddings (both-*OOD* probe ρ): present everywhere (0.84–0.91), no decay with horizon. The teacher-forced rollout falls out of the band by an amount tracking dynamics complexity (horizon-28: 0.84 uniform, 0.57 parabola, 0.24 collision, from 0.98–0.99 at one step); free rollout recovers much of the gap with no physics added.

Two corroborations independent of PhyWorld nMSE.

First, *zero-shot transfer*: on Physion OCP, no synthetic-trained configuration exceeds the random-encoder prior (0.607 mean AUC), and the physics-structured variant is the *worst* of all (0.551, below free rollout’s 0.603)—“more structure, worse transfer,” with most apparent zero-shot competence owed to the architecture, not the learning (Appendix F). Second, *realistic simulation*: trained directly on Physion++ over three seeds, every injection variant we ported—across all three mechanism families (slot, consistency, and the deep-supervision probe)—degrades rollout nMSE 3.6–11.9 $\times$ against the identically trained free-rollout model, worse in *every* seed on all three mass scenarios (the fourth is too baseline-unstable to resolve; Appendix B).

4.3 Mechanism: Load-Bearing but Redundant

The puzzle of §1 localizes here: position is linearly decodable at every horizon, yet the rollout drifts away from it, the further the harder the dynamics (Figure 3). Decodability alone cannot settle what the model *does* with that state (Elazar et al. 2021). We therefore conduct four tests to uncover the effect and mechanism of physical injection.

Test 1: the physical constraint is injected but redundant.

Probing *only the 190 non-slot dimensions* decodes position as well as the full latent in all three domains, undiminished by pinning the slot, while a random-2-dimension control fails (Table 3, top). Thus, the position is injected but a redundant copy.

Test 2: the redundant constraint acts harmfully. Both measurements move (Table 3, bottom): at $\lambda=10$ the weighted physical loss outweighs the prediction loss by one

	uniform	parabola	collision
<i>The redundant copy: position decodability ρ (both-<i>OOD</i>)</i>			
full 192-d latent	0.92	0.85	0.78
black box only (190 d)	0.92	0.85	0.79
black box, slot pinned	0.92	0.86	0.79
the pinned slot itself (2 d)	0.92	0.85	0.51
random-2-d control		0.2–0.5	
<i>The cost: what injection does to the encoder, $\lambda=1 \rightarrow 10$</i>			
loss ratio ($\lambda \mathcal{L}_{\text{phys}}/\mathcal{L}_{\text{pred}}$)	11 $\rightarrow$ 44 $\times$	21 $\rightarrow$ 125 $\times$	3 $\rightarrow$ 15 $\times$
effective dim	41 $\rightarrow$ 4	13 $\rightarrow$ 3	28 $\rightarrow$ 17
collapse	–90%	–77%	–39%

Table 3: Why injection cannot help. *Top*—the 190 non-slot dimensions decode position as well as all 192, undiminished by pinning the slot, while a random-2-d control fails: the position exists in latent space, and an injected slot is a second carrier. *Bottom*—the constraint dominates optimization and collapses the encoder’s participation ratio by 39–90%: strong action on a quantity the latent already encodes.

to two orders of magnitude, and the encoder’s effective dimensionality collapses with it; at $\lambda=1$, the setting of §4.2, the ratio is milder (3–21 $\times$) yet the harm still persists.

Test 3: re-weighting the physical loss does not help.

Sweeping $w=1 \rightarrow 300$ (Appendix B) moves the harm systematically, but weighting rescues only 2 of 4 decision cells, to *parity* and never a net gain: uniform both-*OOD* returns exactly to baseline (0.132 $\pm$ 0.017 vs. 0.136 $\pm$ 0.008), uniform r/m-*OOD* never recovers, collision worsens monotonically. Thus, insufficient physical loss is not the root cause.

Test 4: the injected slot is load-bearing.

We conduct three disjoint interventions to examine whether the injected slot is load-bearing (three seeds, both backbones; excluded cells and full protocol: Appendix D). *Steering* the position the slot decodes moves the model’s own black-box state by 1.5–2.0 $\times$ the offset, while a norm-matched random direction moves it ≈ 0 (additive to within 3%, so the perturbation stays linear). *Patching* the slot with a donor trajectory’s makes prediction follow the donor 47–72% of the way, against 1.5–11% for the same operation on a model with no injected slot. *Jacobians* of predicted position w.r.t. each latent coordinate—which perturb nothing—rank the two injected dimensions 1–2 of 192, up from ~ 99 without injection. From the above results, we conclude that the injected slot is load-bearing, since injecting wrong physical quantities into the slot indeed impairs model performance.

Together, the four tests establish that the injected slot is load-bearing but redundant: it is written into the latent (Test 1) yet duplicates state the black box already carries, so it competes for capacity and gradient without adding new signal (Tests 2–3); the injection sits on the prediction pathway (Test 4), where a mis-specified constraint degrades performance while a correct one yields no gain.

4.4 Free Rollout Reduces Error by 2.2–8.3 $\times$

Replacing teacher forcing with free rollout—a training-protocol change that injects no physics—cuts the hardest-

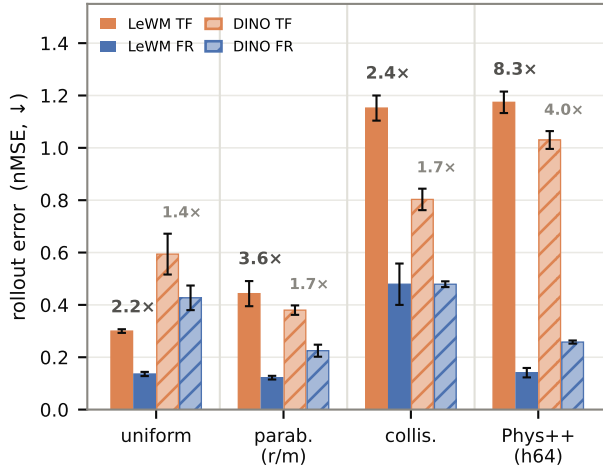


Figure 4: Teacher forcing vs. free rollout, both backbones, grouped by domain—uniform motion, parabola, collision, Physion++ (hardest clean split; nMSE ↓; r/m = radius/mass—OOD split, h64 = horizon 64; solid = LeWM, hatched = frozen DINOv2; blue = free rollout, orange = teacher forcing; three-seed means $\pm$ std, every interval disjoint; per-bar values in Appendix C). Free rollout is worth 2.2–8.3 $\times$ on LeWM and 1.4–4.0 $\times$ on frozen DINOv2—backbone-independent, largest where dynamics are hardest.

split error 2.2–3.6 $\times$ across the three domains and 8.3 $\times$ on Physion++, every interval disjoint over three seeds, and the whole pattern replicates on frozen DINOv2 (Figure 4; Appendix C). The gain is not an OOD patch: *every* partition improves, including ID (2.0–4.6 $\times$). The horizon must match dynamics complexity—collision improves markedly at $H \approx 20$ while smooth domains degrade with excess horizon, and realistic-simulation scenes benefit most from a longer horizon: $H=8 \rightarrow 28$ drives horizon-64 nMSE from 0.280 to 0.014, $\sim 19\times$ (Appendix E).

As the positive control of §3.4, this carries the paper’s methodological weight: on the same code, metrics, seeds, and data that judged all twenty-seven cells, a physics-free intervention produces 2.2–8.3 $\times$ gains—so the injection failure is attributable to the structure, not the pipeline or the yardstick.

4.5 External Control: Removing the Redundancy Is Not Enough

Extrinsic port. The faithful extrinsic port (PIWM; §3.5) learns the correct physics and, where its encoder is in-distribution, edges out our shared-latent model (ρ 0.96/0.97 vs. 0.93/0.87). But under radius/mass shift—which changes appearance, hitting the VAE encoder—it collapses to $\rho=0.33/0.48$ while LeWM holds 0.89/0.87. This is consistent with §4.3 from the outside: making the physical state the *sole, non-redundant* carrier buys a small in-distribution gain that injection into a shared latent never did (their equations alone do not transfer, §4.2)—but it concentrates prediction on one brittle encoder, so removing the redundancy trades

the duplicated-state problem for an encoder-OOD one instead of solving it. An extrinsic sole-carrier design is not, by itself, a fix.

Cross-backbone replication. On the frozen-DINOv2 variant every primary finding replicates across three seeds. Presence is even stronger—the physics-naïve frozen encoder decodes position at $\rho=0.95$ (vs. 0.90 for LeWM), with the same redundant black-box copy—so both the state and its duplication are properties of generic visual features, not of our training. The protocol lever transfers (1.4–4.0 $\times$; Figure 4); injection still never wins (26 of 27 at or below baseline; the one below-cell separates at 0.97 $\times$ by a 0.001 margin), and the apparent uniform gains dissolve under content-free controls into capacity-restriction regularization, not physics (Trap 5, Appendix F). One difference: with the encoder frozen, injection is largely inert rather than harmful—the losses reshape only the adapter, bounding damage but foreclosing benefit. Across every control the same sentence survives: physical state in a latent world model is *load-bearing but redundant*.

5 Conclusion

We asked whether injecting physics can make a latent world model physically consistent, and answered with a controlled anatomy: injection fails in 26 of 27 cells—with the exact gravitational form, with perfect labels, grafted on or co-trained from scratch. The account we defend is that the state is already there: the non-slot dimensions carry the same position, so an injected copy adds no signal while the constraint consumes capacity and gradient—*load-bearing but redundant*. Intervention settles the half that decodability alone leaves open, and against the reading Tests 1–3 invite: the injected slot *is* read, by three methods that fail in different ways, while the black box keeps its own copy. Re-weighting reaches only parity; an architecture that removes the redundancy does help, until an OOD shift reaches its encoder—not, by itself, a fix. What moved prediction was never the structure but the training protocol, worth 2.2–8.3 $\times$ on identical code and metrics.

The constructive reading: an inductive bias must supply what the latent is missing—*how the state evolves, not what the state is*. Two routes stay open: injections that target dynamics or conserved quantities rather than state, and extrinsic designs paired with OOD-robust encoders. Our own evolution-targeting variants are deliberately simple—a kinematic update and a consistency prior on slot coordinates—so their failure bounds that class, not evolution injection in general: collision turns on contact and inter-object relation, which only operator-level designs (symplectic, learned contact) can express.

Limitations. Our conclusions rest mainly on one compact backbone (~ 10 M ViT-tiny JEPa); the primary findings replicate on frozen DINOv2, where injection is largely inert rather than harmful—so the *harm* claim is established on the trainable-encoder regime. On the mechanism, the slot’s use is demonstrated; the black box’s role stays partly inferential—erasing its position copy takes 116–154 of 190

dimensions, and at that rank no valid same-rank control exists—a rank-matched random projection removes $2.9\text{--}4.7\times$ more of the black box’s variance than the position ablation does, so the comparison reports how much each removed rather than which information, and matching the variance is geometrically impossible there (a limit we measure, not a step we skipped)—while wholesale replacement passes its controls in one of six cells.

References

- Adebayo, J.; Gilmer, J.; Muelly, M.; Goodfellow, I.; Hardt, M.; and Kim, B. 2018. Sanity Checks for Saliency Maps. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Alain, G.; and Bengio, Y. 2016. Understanding intermediate layers using linear classifier probes. arXiv:1610.01644.
- Assran, M.; Duval, Q.; Misra, I.; Bojanowski, P.; Vincent, P.; Rabbat, M. G.; LeCun, Y.; and Ballas, N. 2023. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*, 15619–15629.
- Balestriero, R.; and LeCun, Y. 2025. LeJEPa: Provable and Scalable Self-Supervised Learning Without the Heuristics.
- Bardes, A.; Garrido, Q.; Ponce, J.; Chen, X.; Rabbat, M.; LeCun, Y.; Assran, M.; and Ballas, N. 2024. Revisiting Feature Prediction for Learning Visual Representations from Video. *Trans. Mach. Learn. Res.*, 2024.
- Battaglia, P. W.; Pascanu, R.; Lai, M.; Rezende, D. J.; and Kavukcuoglu, K. 2016. Interaction Networks for Learning about Objects, Relations and Physics. In *Advances in Neural Information Processing Systems 29 (NIPS)*, 4502–4510.
- Bear, D.; Wang, E.; Mrowca, D.; Binder, F. J.; Tung, H.; Pramod, R. T.; Holdaway, C.; Tao, S.; Smith, K. A.; Sun, F.; Li, F.; Kanwisher, N.; Tenenbaum, J.; Yamins, D.; and Fan, J. E. 2021. Physion: Evaluating Physical Prediction from Vision in Humans and Machines. In Vanschoren, J.; and Yeung, S., eds., *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*.
- Bengio, S.; Vinyals, O.; Jaitly, N.; and Shazeer, N. 2015. Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks. In Cortes, C.; Lawrence, N. D.; Lee, D. D.; Sugiyama, M.; and Garnett, R., eds., *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*, 1171–1179.
- Cranmer, M.; Greydanus, S.; Hoyer, S.; Battaglia, P.; Spergel, D.; and Ho, S. 2020. Lagrangian Neural Networks. ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations. arXiv:2003.04630.
- Elazar, Y.; Ravfogel, S.; Jacovi, A.; and Goldberg, Y. 2021. Amnesic Probing: Behavioral Explanation with Amnesic Counterfactuals. *Transactions of the Association for Computational Linguistics*, 9: 160–175.
- Goyal, A.; Lamb, A.; Zhang, Y.; Zhang, S.; Courville, A. C.; and Bengio, Y. 2016. Professor Forcing: A New Algorithm for Training Recurrent Networks. In Lee, D. D.; Sugiyama, M.; von Luxburg, U.; Guyon, I.; and Garnett, R., eds., *Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5-10, 2016, Barcelona, Spain*, 4601–4609.
- Greydanus, S.; Dzamba, M.; and Yosinski, J. 2019. Hamiltonian Neural Networks. In Wallach, H. M.; Larochelle, H.; Beygelzimer, A.; d’Alché-Buc, F.; Fox, E. B.; and Garnett, R., eds., *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, 15353–15363.
- Ha, D.; and Schmidhuber, J. 2018. Recurrent World Models Facilitate Policy Evolution. In Bengio, S.; Wallach, H. M.; Larochelle, H.; Grauman, K.; Cesa-Bianchi, N.; and Garnett, R., eds., *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, 2455–2467.
- Hafner, D.; Pasukonis, J.; Ba, J.; and Lillicrap, T. 2025. Mastering diverse control tasks through world models. *Nature*, 640: 647–653.
- Hewitt, J.; and Liang, P. 2019. Designing and Interpreting Probes with Control Tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2733–2743.
- Kang, B.; Yue, Y.; Lu, R.; Lin, Z.; Zhao, Y.; Wang, K.; Huang, G.; and Feng, J. 2025. How Far is Video Generation from World Model: A Physical Law Perspective. In *International Conference on Machine Learning (ICML)*.
- LeCun, Y. 2022. A Path Towards Autonomous Machine Intelligence. OpenReview preprint.
- Liu, Z.; Sanborn, S.; Ganguli, S.; and Tolia, A. 2026. From Kepler to Newton: Inductive Biases Guide Learned World Models in Transformers. arXiv:2602.06923.
- Locatello, F.; Weissenborn, D.; Unterthiner, T.; Mahendran, A.; Heigold, G.; Uszkoreit, J.; Dosovitskiy, A.; and Kipf, T. 2020. Object-Centric Learning with Slot Attention. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*.
- Maes, L. 2025. LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. GitHub repository.
- Mao, Z.; Umasudhan, M. E.; and Ruchkin, I. 2024. Physically Interpretable World Models via Weakly Supervised Representation Learning.
- Nam, H.; Lidec, Q. L.; Maes, L.; LeCun, Y.; and Balestriero, R. 2026. Causal-JEPa: Learning World Models through Object-Level Latent Interventions. In *International Conference on Machine Learning (ICML)*.
- Nie, W.; Liu, W.; Guo, H.; and Su, Y. 2026. Phys-JEPa: Physics-Informed Latent World Models for Multivariate Time-Series Forecasting. arXiv:2606.16076.

Oquab, M.; Darcet, T.; Moutakanni, T.; Vo, H. V.; Szafraniec, M.; Khalidov, V.; Fernandez, P.; Haziza, D.; Massa, F.; El-Nouby, A.; Assran, M.; Ballas, N.; Galuba, W.; Howes, R.; Huang, P.-Y.; Li, S.-W.; Misra, I.; Rabbat, M.; Sharma, V.; Synnaeve, G.; Xu, H.; Jégou, H.; Mairal, J.; Labatut, P.; Joulin, A.; and Bojanowski, P. 2024. DI-NOv2: Learning Robust Visual Features without Supervision. *Transactions on Machine Learning Research*.

Peper, J.; Mao, Z.; Geng, Y.; Pan, S.; and Ruchkin, I. 2025. Four Principles for Physically Interpretable World Models. In *Proceedings of the International Conference on Neuro-symbolic Systems (NeuS)*, volume 288 of *Proceedings of Machine Learning Research*, 66–89.

Raissi, M.; Perdikaris, P.; and Karniadakis, G. E. 2019. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378: 686–707.

Ravfogel, S.; Elazar, Y.; Gonen, H.; Twiton, M.; and Goldberg, Y. 2020. Null It Out: Guarding Protected Attributes by Iterative Nullspace Projection. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Sanchez-Gonzalez, A.; Godwin, J.; Pfaff, T.; Ying, R.; Leskovec, J.; and Battaglia, P. W. 2020. Learning to Simulate Complex Physics with Graph Networks. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*.

Simonyan, K.; Vedaldi, A.; and Zisserman, A. 2013. Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps. *arXiv preprint arXiv:1312.6034*.

Sundararajan, M.; Taly, A.; and Yan, Q. 2017. Axiomatic Attribution for Deep Networks. In *International Conference on Machine Learning (ICML)*.

Tung, H.; Ding, M.; Chen, Z.; Bear, D.; Gan, C.; Tenenbaum, J.; Yamins, D.; Fan, J. E.; and Smith, K. A. 2023. Physion++: Evaluating Physical Scene Understanding that Requires Online Inference of Different Physical Properties. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Venkatraman, A.; Hebert, M.; and Bagnell, J. A. 2015. Improving Multi-Step Prediction of Learned Time Series Models. In Bonet, B.; and Koenig, S., eds., *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, January 25-30, 2015, Austin, Texas, USA*, 3024–3030.

Zahorodnii, A. 2025. Improving World Models using Deep Supervision with Linear Probes. ICLR 2025 Workshop on World Models.

Zhou, G.; Pan, H.; LeCun, Y.; and Pinto, L. 2025. DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning. In *International Conference on Machine Learning (ICML)*.

A Training and Evaluation Details

Reproducibility. All training configurations, per-run logs, evaluation protocols, and the exact numbers behind every table and figure are archived alongside the code; the study builds on public assets (LeWM, PhyWorld, Physion, Physion++). We will release the evaluation suite, including the per-horizon/per-partition audit tools and the random-encoder controls.

Hyperparameters. All PhyWorld runs: AdamW, lr 5×10^{-5} (from-scratch reruns: 2×10^{-5}), weight decay 10^{-3} , bf16, gradient clip 1.0, 20 epochs; the from-scratch cells of the pretraining-regime 2×2 train longer and at a lower learning rate so their baselines converge cleanly—60 epochs on uniform motion and collision, 120 on parabola, which needed the extra budget—with injection on and off sharing the identical budget within each domain, so every Δ in Table 2 is a within-domain comparison. Batch size 64 for free rollout ($H=8$), 128 for teacher forcing ($H=1$), 48 for $H \geq 16$. History size 3; embedding dimension 192; SIGReg weight 0.09. Seeds: 3072/1234/42 where seed-replicated; both arms of the pretraining-regime 2×2 are three-seed (Table 2): the from-scratch cells were replicated directly, and the fine-tune cells are read off the three-seed scan of Table 4, which is the same regime. Physion++ runs use the readout split (800 clips) with the same optimizer.

OOD partitions. ID: radius/mass $\in [0.7, 1.5]$, velocity $\in [1.0, 4.0]$. OOD draws radius/mass from $[0.3, 0.6] \cup [1.5, 2.0]$ and velocity from $[0, 0.8] \cup [4.5, 6.0]$. Collision uses the four-column variant (two objects’ masses and velocities). Training sets are ID-only (1,000 trajectories); evaluation covers all four partitions ($\geq 1,600$ trajectories per domain).

Probe protocol. Ridge regression ($\alpha=1$), trajectory-level 80/20 splits, probing the encoder output (not the projector), pretrained “paper” initialization, and $K=4$ stacked consecutive latents for velocity. Each choice matters: the naive combination (random init, single frame, through-projector) reads $\rho=0.166$ for uniform velocity where the corrected protocol reads 0.939 (Trap 4, Appendix F). Random-encoder and pixel-statistics controls are run alongside every probe.

From-scratch 2×2 , per seed. Hardest clean split, latent nMSE (uniform/collision both-OOD, parabola r/m-OOD), seeds 3072/1234/42: uniform off 0.192/0.230/0.246, on 0.750/0.560/0.662; parabola off 0.343/0.420/0.666, on 0.375/0.494/0.473; collision off 0.538/0.513/0.454, on 0.635/0.661/0.649. The parabola from-scratch baseline spans 0.343–0.666 across seeds—a spread wider than the effect being measured, which is why that cell is reported as parity rather than as a gain.

B Full Injection Results

Table 4 is the numeric form of Figure 2, with the seed annotations spelled out.

Probe/slot factorial (uniform, free rollout). Latent both-OOD nMSE / pixel both-OOD PSNR (dB), seed-3072 values: baseline 0.131 / 20.41; probe-only 0.167 / 19.83; slot+reweight 0.114 / 21.30; probe+slot+reweight 0.125

	uniform (both-OOD)	parabola (r/m-OOD)	collision (both-OOD)
Free-rollout baseline	0.136	0.122	0.479
Fixed slot	0.162°	0.142	0.625
+reweight ($w=30$)	0.132°	0.121°	0.614
+velocity	0.185	0.096	0.630
Probe	0.185	0.137°	0.657
+slot+reweight	0.141°	0.099°	0.623
Kinematic (free MLP)	0.155	0.149°	0.557°
Kinematic $a=g$ (= grounded)	0.173	0.151	0.536°
Consistency	0.143°	0.151	0.610
Label-free prior	0.169	0.120°	0.643

Table 4: The 27-cell scan (hardest clean split, latent nMSE ↓; every cell and the baseline are three-seed means over seeds 3072/1234/42). **Bold** marks the one seed-robust gain (velocity+slot on parabola: 0.096 ± 0.007 vs. baseline 0.122 ± 0.007 , intervals disjoint). ° marks cells within noise—their seeds overlap the baseline’s; unmarked cells are worse in every seed. 15 cells worse, 11 within noise, 1 better. The $a=g$ row doubles as the grounded prior: adding labels to the label-free structure yields exactly the strict $a=g$ specification, and the configuration was in fact launched under both names before the identity was noticed. The row therefore pools four draws per domain (two independent seed-3072 runs plus the 1234/42 top-ups); the two 3072 draws bracket the mean (uniform 0.206 and 0.166), so the extra weight on that seed does not bias the cell.

/ 20.02. The combination underperforms the slot alone on both scales; the latent and pixel verdicts agree.

Slot reweighting sweep (uniform). Slot weight 1/30/100: 0.183 / 0.114 / 0.136 (single seed), with the $w=30$ cell seed-replicated at 0.132 ± 0.017 against the baseline 0.136 ± 0.008 . Kinematic head on the reweighted slot: velocity-OOD decoded position $\rho=0.991/0.987$ (x/y); uniform pixel horizon-28 23.34 vs. 22.09 dB (+1.25); appearance-OOD remains the weak axis (decoded ρ $0.29 \rightarrow 0.43$, vs. 0.96 without the kinematic head).

Redundancy probe details. Protocol for §4.3, which reports the numbers. Two models are probed: the free-rollout baseline (no injection) and the slot-pinned model (structpos, $w=30$). For each, a ridge probe is fit on the dimension subset alone—the full 192, the black box [2:192], the slot [0:2], or a random pair as control—with train/test trajectories split 80/20, so decodability is measured out of sample. Values are Pearson ρ against ground-truth position on the both-OOD partition, averaged over both coordinates, from frame embeddings; the same table holds on rolled-out latents (Appendix D). The slot absorbs position (its supervision loss falls $2.53 \rightarrow 0.015$) without displacing the black box’s redundant copy.

Encoded quantity vs. domain (position monotone, velocity sign-flipping). The reweighted slot’s effect is systematic in what it encodes (hardest clean split, legacy seed-3072 values; the three-seed baselines are 0.136/0.122/0.479, Ta-

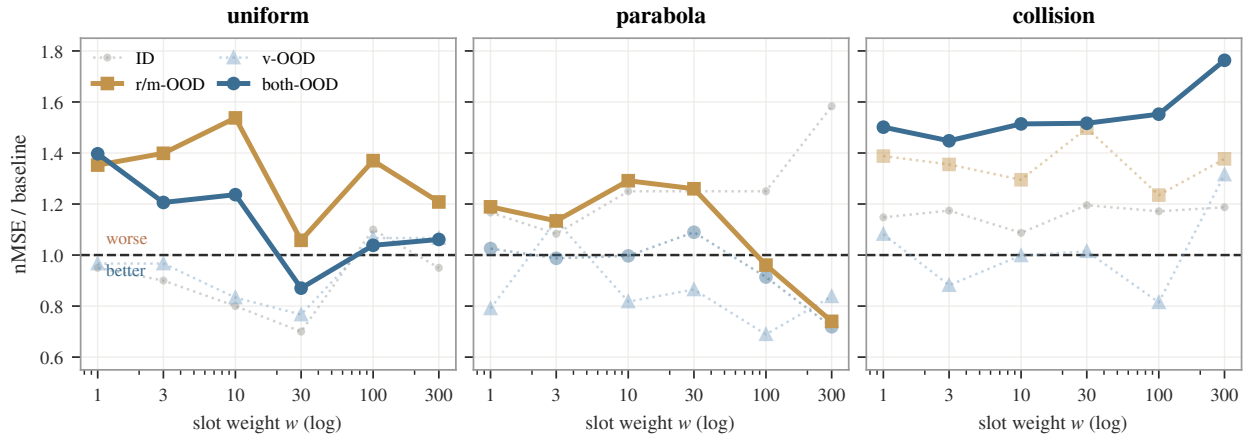


Figure 5: Dose-response re-weighting, all partitions. Solid curves are the four decision cells (hardest clean split); dotted are partitions we do not judge on—ID is not an extrapolation test, and parabola’s v-OOB/both-OOB nMSE is denominator-degenerate (Appendix F). Only 2 of the 4 decision cells reach parity (uniform both-OOB, parabola r/m-OOB); uniform r/m-OOB and collision both-OOB stay above baseline at every weight, and collision degrades as weight grows. The sweep is single-seed; both parity cells were re-run over three seeds and land at baseline (§4.3).

ble 4):

slot content	uniform	parabola	collision
position (structpos)	0.132	0.160	0.596
position+velocity	0.207	0.096	0.621
Δ (adding velocity)	+0.075	−0.064	+0.025

Position is an output variable with the same role in every domain, so pinning it degrades monotonically with dynamics complexity and never flips sign. The marginal effect of adding velocity (+0.075/−0.064/+0.025) tracks velocity’s role in each domain—redundant constant (uniform), linear driver (parabola), discontinuous jump (collision). The parabola gain ($0.122 \pm 0.007 \rightarrow 0.096 \pm 0.007$, every seed lower) is thus the sole seed-robust structural gain in the grid: a fixed form landing on a matching domain, not a portable prior.

Physion++ per-scenario (all ten scenarios, FR at 20 epochs). Rollout nMSE: bouncy_platform 0.041, bouncy_wall 0.094, friction_cloth 0.071, friction_collision 0.062, friction_platform 0.041, mass_collision 0.083, mass_dominos 0.058, mass_waterpush 0.122; deformable scenes are the common hard case for every method (deform_clothhang 1.375, deform_clothhit 0.772). Injection variants—slot, consistency, and probe alike—degrade the mass scenarios by 3.6–11.9 $\times$ in every seed (Table 5) while leaving the deformable failure in place.

Physion++ horizons. Overall nMSE at horizons 16/32/64: baseline ($H=8$) 0.016/0.140/0.280; $H=20$ 0.022/0.033/0.220; $H=16$ +scale 0.010/0.035/0.136; $H=20$ +scale 0.012/0.024/0.087; $H=28$ +scale reaches horizon-64 nMSE 0.014. Free rollout is the main long-horizon lever (horizon-32 cosine 0.91 vs. 0.79–0.87 for every physics-augmented variant); geometric scale jitter acts as an amplitude stabilizer for long rollouts: without it,

$H=20$ explodes on friction-collision (nMSE 24.6 at cosine 0.99); with it, 0.019.

C Cross-Backbone Replication: Frozen DINOv2

The cross-backbone variant (§3.5) freezes a DINOv2-small encoder (Oquab et al. 2024)—in the style of DINO-WM (Zhou et al. 2025)—and trains only the projector (as adapter) and the identical predictor stack, losses, and seeds (~ 130 runs). Free-rollout baselines (three-seed): uniform both-OOB 0.427 ± 0.039 , parabola r/m-OOB 0.225 ± 0.019 , collision both-OOB 0.479 ± 0.009 ; Physion++ horizon-64 0.258 ± 0.005 vs. teacher forcing 1.030 ± 0.028 .

Teacher forcing vs. free rollout, per domain. Values behind Figure 4 (hardest clean split, latent nMSE $\downarrow$, three-seed mean $\pm$ sample std). LeWM: uniform $0.300 \pm 0.007 \rightarrow 0.136 \pm 0.008$ (2.2 $\times$), parabola $0.443 \pm 0.048 \rightarrow 0.122 \pm 0.007$ (3.6 $\times$), collision $1.152 \pm 0.048 \rightarrow 0.479 \pm 0.079$ (2.4 $\times$), Physion++ horizon-64 $1.174 \pm 0.041 \rightarrow 0.141 \pm 0.018$ (8.3 $\times$). Every teacher-forcing interval is disjoint from its free-rollout counterpart. The frozen-DINOv2 counterparts above give 1.4–4.0 $\times$ over the same four settings, so the lever survives replacing a trainable ViT-tiny with a frozen general-purpose encoder.

Presence and redundancy. The frozen encoder, which has never seen PhyWorld and received no physics supervision or gradient, decodes position from real-frame embeddings at $\rho=0.951$ (both-OOB)—higher than the PhyWorld-trained LeWM encoder (0.899). The 190 non-slot dimensions alone decode at 0.951, indistinguishable from the full latent; pinning the slot at weight 300 leaves the black-box copy at 0.973. Presence and the redundant copy are properties of generic visual representations.

Scenario	FR	+slot	+cons.	+c.+acc.	+probe	+pr.slot
friction_platform ^o	0.220±0.231	0.160±0.059	0.171±0.120	0.134±0.058	0.212±0.041	0.208±0.025
mass_collision	0.171±0.079	0.616±0.112	0.725±0.100	0.726±0.212	0.666±0.077	0.864±0.112
mass_dominoes	0.069±0.013	0.424±0.122	0.585±0.021	0.478±0.139	0.320±0.055	0.683±0.182
mass_waterpush	0.122±0.002	0.440±0.073	0.591±0.064	0.780±0.609	0.510±0.048	0.479±0.170

Table 5: Physion++ direct training: per-scenario rollout nMSE ($\downarrow$) at 20 epochs, three-seed mean±std (3072/1234/42). On the three mass scenarios every injection variant—spanning all three mechanism families (slot, consistency, probe)—is worse in *every* seed, the baseline’s worst draw beating the best injected draw, degrading prediction 3.6–11.9 $\times$. ^ofriction_platform is unresolved and we draw no conclusion from it: its own baseline spans 0.041–0.480 across seeds, a 12 $\times$ spread that swamps the effect, and on the mean the injected arms sit *below* it. The single-seed version of this table read that cell as a 3 $\times$ degradation; three seeds show it was seed luck. Deformable-object scenes remain the hardest category for every method including the baseline.

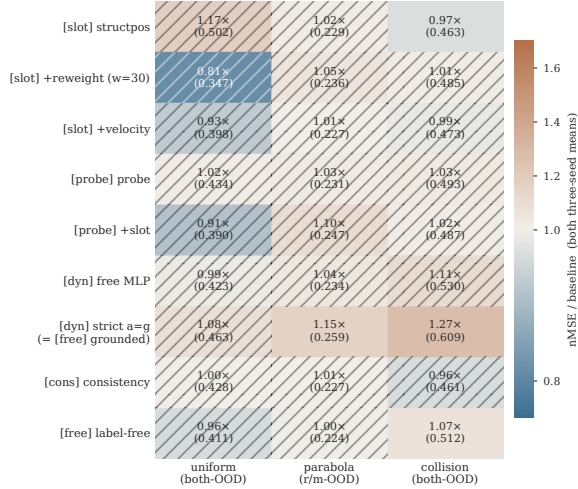


Figure 6: The injection scan repeated on the frozen-DINOv2 backbone, drawn and judged exactly as Figure 2 (three-seed means over three-seed baseline; hatched = the cell’s seeds overlap the baseline’s; the $a=g$ /grounded row merged, four draws). 3 cells worse in every seed / 23 within noise / 1 below (26 of 27 at or below baseline; the one below-cell separates by a 0.001 margin at 0.97 $\times$). The dynamics-head row is the worst here too (1.27 $\times$ on collision); the visually blue uniform cells are all within noise and additionally dissolve under the content-free controls of Trap 5.

Inert, not harmful. Unlike the trainable-encoder regime, almost everything lands within the baseline’s seed band (23 of 27 cells): the only cells that separate as worse are the dynamics-head row (1.15 $\times$ parabola, 1.27 $\times$ collision) and the label-free prior on collision (1.07 $\times$). With the encoder frozen, injection losses can only reshape the 384→192 adapter: the redundant copy cannot be displaced, which bounds the damage—and forecloses the benefit. Injection-variant variance also roughly doubles (± 0.087 vs. baseline ± 0.039 on uniform), so single-seed cells on this backbone are uninformative.

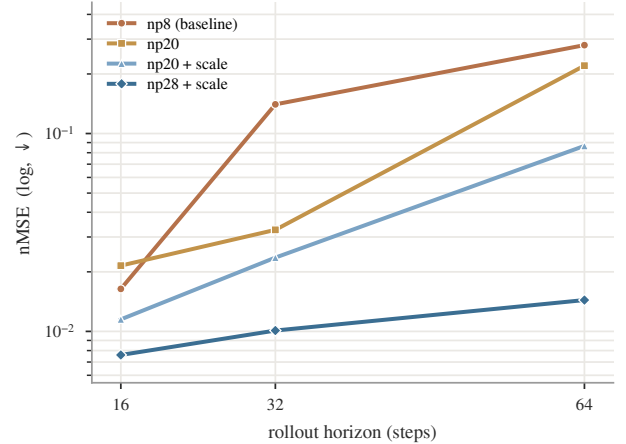


Figure 7: Physion++ direct training: per-horizon rollout nMSE (log scale) as the training horizon grows ($H=8 \rightarrow 28$, +scale jitter). Long-horizon error falls monotonically to 1/19 of baseline with no inflection: realistic-simulation dynamics reward long-rollout training more than the synthetic domains do.

D Causal Interventions on Trained Models

Protocol and full results for Test 4 (§4.3). All three interventions run at inference on existing checkpoints—no retraining—over three seeds (3072/1234/42), three domains, and both backbones. Position is read with the same ridge probe used elsewhere, fit on the training split.

Why the readout is the black box. Each intervention perturbs one channel and reads position off the *other*. Reading back the channel just written is partly trivial: the predictor copies its input forward, so the slot readout moves whether or not the perturbation reached the computation. The black-box readout is the one that reflects it (it is also what drives the pixel decoder). Predictions are recorded *before* the per-step perturbation is applied, so every number below is read off what the model itself produced.

Steering. For a target offset δ (in units of position std), we add the minimum-norm latent offset that moves the named channel’s decoded position by δ —for the supervised slot,

writing $\text{pos}+\delta$ directly, which is better conditioned—and report $g = \Delta(\text{decoded position})/\delta$. Slot gains on the black-box readout, three-seed mean $\pm$ std, against a norm-matched random direction in the black box:

backbone	domain	g_{slot}	random control
LeWM	uniform	$+1.52\pm 0.03$	-0.017
LeWM	collision	$+2.01\pm 0.20$	-0.003
LeWM	parabola	$+1.15\pm 1.40$	-0.017
DINOv2	uniform	$+1.69\pm 0.06$	-0.011
DINOv2	parabola	$+0.59\pm 0.30$	$+0.001$
DINOv2	collision	$+1.94\pm 0.31$	-0.031

Additivity holds to within 3% ($g_{\text{slot}}+g_{\text{bb}}$ vs. steering both together), so the perturbation stays in the linear regime. *The magnitudes are not proportions*: values above 1 reflect probe $R^2 < 1$, an unperturbed action input, and the 3-frame history anchor. They support “used vs. not” and “this channel vs. that one”, nothing quantitative. LeWM/parabola is excluded: its seeds read $-0.10/+0.90/+2.66$, a spread wider than the effect, and a $20\times$ sweep of δ leaves the per-seed values essentially unchanged, so this is three different models rather than one noisy measurement.

Counterfactual patching. We replace the slot with a donor trajectory’s slot at the same timestep, leaving the black box at its own values, and regress the decoded position on [own truth, donor truth]; the follow-fraction is the donor’s share. The control is the same operation on a model with no injected slot, where those two dimensions carry no physical meaning: LeWM/uniform $0.015 \rightarrow 0.468$, DINOv2/uniform $0.112 \rightarrow 0.650$, DINOv2/collision $0.091 \rightarrow 0.721$. The LeWM parabola and collision controls are not clean (a no-slot model follows $0.42\text{--}0.51$), and we exclude those cells; a collinearity explanation fits parabola (pooled own–donor correlation 0.76) but not collision (0.22), so the cause is unresolved.

Per-dimension Jacobians. $J[c, j] = \partial p_c / \partial z_j$, the sensitivity of decoded position p_c to latent coordinate z_j , averaged over 400 frames, reported per dimension so a 2-d slot and a 190-d black box share a scale. Gradient sensitivity is standard (Simonyan, Vedaldi, and Zisserman 2013; Sundararajan, Taly, and Yan 2017), here applied to an intermediate representation rather than to pixels. Unlike the interventions, it chooses no direction, which matters because a redundant code barely moves along any single one; all six cells are usable here where the black-box replacement below leaves one. Its known weakness is the reason we do not rest on it alone: a gradient is local and linear, so it reports what a small change would do rather than what changing the channel does, and some saliency methods are insensitive to the model they purport to explain (Adebayo et al. 2018). The baseline rows answer that second concern directly—the readout tracks whether the model was injected, not the input structure—and the interventions above answer the first. The baseline rows are a built-in control: with no injection those two dimensions are arbitrary and should rank mid-pack, and they do (ratio $0.80\text{--}0.98$, ranks $\sim 90\text{--}162$ of 192). After injection they rank 1st–2nd on LeWM uniform/parabola and DINOv2 uniform, at $2.3\text{--}4.0\times$ the per-dimension sensitivity

of the black box; DINOv2/parabola is the exception (ratio 0.99 , ranks $111/90$). The slot nonetheless holds only $0.8\text{--}4.0\%$ of *total* sensitivity—two dimensions against 190—which is the analytic form of “both channels carry load”.

Amnesic projection: a limit we quantify. We remove the linearly position-predictive subspace from the black box by iterative nullspace projection (Ravfogel et al. 2020), with a rank-matched random-ablation control. Erasing position requires removing 116–154 of the 190 dimensions—the hardest quantification of redundancy in this paper, and far stronger than the random-2-dimension control of Table 3. But at that rank the test cannot discriminate: in five of six cells, removing position damages the rollout *less* than removing the same number of dimensions at random (e.g. uniform structpos 0.608 vs. 1.116). The reason is quantitative, and we measured it rather than leaving it as a caveat. INLP selects linearly predictive directions, which in this representation are also low-variance ones: erasing position removes only $0.17\text{--}0.27$ of the black box’s variance, while a rank-matched random projection removes $0.77\text{--}0.82$ —a mismatch of $2.9\text{--}4.7\times$ (mean $3.8\times$) across the six cells. The rank-matched control therefore destroys several times more of the representation than the ablation it is meant to calibrate, and the comparison mostly reports how much variance each removed rather than which information. Nor can the mismatch be repaired at this rank: searching random rank-matched subspaces for the one closest in removed variance still lands at $0.74\text{--}0.77$, since removing ~ 150 of 190 random directions cannot take only a fifth of the variance. Under that variance-matched control the same pattern holds in six of six cells. So the honest reading is not that we chose a weak control but that at this rank no valid same-rank control exists—itsself a consequence of how distributed the copy is. Per cell (INLP / rank-matched random / variance-matched random, as fractions of the black box’s total variance): uniform baseline $0.27/0.78/0.75$, uniform structpos $0.17/0.82/0.77$, parabola baseline $0.21/0.77/0.74$, parabola structpos $0.22/0.78/0.77$, collision baseline $0.25/0.81/0.77$, collision structpos $0.18/0.79/0.77$; INLP rank 146–154 of 190 throughout.

One further caveat we state rather than smooth over: an earlier run stopped at 24 removed dimensions showed position-removal damaging the rollout *less* than random removal—apparent support for a bypass—a contrast that disappears once INLP is run to convergence, and which the variance account explains, the mismatch being far smaller at low rank.

Black-box replacement. Replacing k black-box dimensions outright with a donor’s (slot untouched) sidesteps the min-norm problem: on a baseline model, where the black box is the only route position can take, the min-norm patch yields a follow-fraction of 0.089 while replacing all 190 yields 0.961 —an order of magnitude apart on the same model and channel. Sweeping k gives the crossing point at which moving the black box buys as much as moving the 2-d slot. A cell is usable only if its baseline both ignores a patch to the two dimensions it does not have and follows the donor once the whole black box is replaced; only LeWM/uniform

passes (0.015 and 0.961), where the crossing sits near 100 dimensions. The remaining five cells fail one or both controls and their crossings are not quoted.

E Augmentation: a Domain-Specific Lever with a Reversal Boundary

Augmentation is not part of the paper’s main line (it is neither an injection mechanism nor a protocol variable), but its boundary behavior is a useful caution. On synthetic domains, per-trial appearance jitter (brightness/contrast, strength 0.5) roughly halves the hardest split: uniform $0.131 \rightarrow 0.068$ (−48%), parabola −63% (r/m-ODD, seed 3072: $0.127 \rightarrow 0.065$); geometric scale jitter, only in combination with a long horizon, sets the collision record (0.172 ± 0.004 over three seeds, vs. the 0.479 ± 0.079 free-rollout baseline). Interactions are non-monotone: appearance conflicts with long-horizon training (0.472, worse than either alone), scale synergizes (0.208), the triple is worse than the best pair (0.253); temporal-stride augmentation loses even to appearance on velocity-ODD (0.556 vs. 0.376), leaving unseen velocities the open hard case.

The appearance recipe reverses on realistic simulation: the same jitter that halves synthetic OOD error degrades Physion++ friction-collision prediction by two orders of magnitude (nMSE $0.062 \rightarrow 6.44$) while the *aggregate* long-horizon cosine improves (Trap 1 below), and it does not help zero-shot transfer either (0.597, below the 0.607 random-prior ceiling). The boundary is principled: appearance jitter encodes an appearance-invariance assumption that holds under PhyWorld’s simple renderer but breaks where appearance carries physical information (friction, mass, material). Scale jitter, which assumes only size-invariance, keeps helping at realistic-simulation scale (Appendix B); appearance jitter flips sign. Augmentation gains are domain- and type-specific, not portable.

F Metric Pathologies and Evaluation Traps

Our anatomy repeatedly reached conclusions opposite to those suggested by common metrics; the decision rules of §4.1 distill the following five failure modes, each of which reversed at least one finding.

Trap 1: cosine and probe metrics are duals of the training loss. Probe correlation and latent cosine rise mechanically when probe or slot losses are optimized—they measure whether information is *present*, not whether prediction *uses* it. Probe supervision lifts velocity decodability from $\rho=0.44$ to 0.80 and gives the most stable long-horizon *cosine* (0.882 vs. 0.843), while its *pixel* rollout is the worst in the comparison (19.93 vs. 20.64 dB at horizon 28). Appearance augmentation on Physion++ improves the aggregate long-horizon cosine while per-scene nMSE collapses up to $100\times$ (deform_clothhit: cosine $0.610 \rightarrow 0.913$, nMSE $0.772 \rightarrow 3.69$). Judging either result by direction-only or readout metrics inverts the conclusion. Our own early sweep was misled this way: judged on $K=4$ probe ρ , $\lambda=50$ “won”; re-judged on prediction loss, the ranking reversed.

Trap 2: nMSE denominators can degenerate. When the target sequence degenerates (a parabola ball exits the frame, target variance $\rightarrow 0$), per-horizon nMSE explodes— $3 \times 10^4 - 2 \times 10^6$ at horizon ~ 28 across all six parabola baseline variants, while cosine sits at a healthy 0.55–0.95—and pollutes every aggregate that includes late horizons. This is why parabola is judged on its clean r/m-ODD partition throughout. Traps 1 and 2 point in opposite directions (cosine under-reports failure; degenerate nMSE over-reports it); neither metric family is safe alone, hence per-partition, per-horizon cross-validation.

Trap 3: zero-shot transfer is bounded by the architecture prior. On Physion OCP, a *randomly initialized* encoder scores mean AUC 0.607. No synthetic-training configuration exceeds it: free rollout approaches it (0.603), physics variants fall below it (0.551–0.582), appearance augmentation 0.579–0.597; training longer approaches the ceiling monotonically (0.554 at epoch 1 to 0.603 at epoch 20) without crossing. Per-scenario, only Support (+0.10) and Collide (+0.07) show signal above the random control. Zero-shot claims must be calibrated against the random-prior floor, or they measure the architecture, not the learning.

Trap 4: protocol confounds fabricate results in both directions. Three probe-protocol choices (random vs. pre-trained initialization, single-frame vs. stacked probes, probing through the projector) multiply into a false negative: the naive stack reads uniform velocity decodability at $R^2=0.166$, the corrected protocol 0.939. A silent initialization bug (only 24 of 216 loadable encoder keys actually loaded) invalidated an entire 45-configuration sweep before a loading guard caught it. And converged training loss proves nothing about rollout: our clean from-scratch baseline converges to the pretrained model’s prediction loss (0.006 vs. 0.005) while its rollout OOD error is $2.8\times$ worse.

Trap 5: loss re-weighting masquerades as physics. On the frozen-DINOv2 backbone, heavily re-weighting the pinned position slot (weight 300; the 2 slot dimensions then carry 76% of the prediction loss) improves uniform both-ODD from 0.427 ± 0.039 to 0.286 ± 0.015 —apparently the one clear injection win on either backbone. Two content-free controls dissolve it: pinning the slot to a *shuffled*, physically meaningless target at the same weight reaches 0.330 ± 0.013 , and weakening the pin $10\times$ while keeping the weighting reaches 0.329 ± 0.084 —most of the gain, with no usable physics. All three variants degrade in-distribution error ($0.099 \rightarrow 0.115$ –0.136), the signature of capacity restriction (the predictor is effectively reduced to a 2-dimensional task), and the same recipe *worsens* collision ($1.11\times$) and parabola ($1.14\times$). An apparent injection gain must be checked against a content-free control at matched loss weighting.

Auxiliary-loss dominance (measurement behind §4.3, Test 2). At probe weight $\lambda=10$, the weighted auxiliary-to-prediction loss ratios—used as a proxy for gradient magnitude; we do not measure gradient norms directly—are $15\times$ (collision), $44\times$ (uniform), and $125\times$ (parabola); the encoder’s effective (participation-ratio) dimensionality collapses by 39–90% relative to baseline (uniform $41 \rightarrow 4$). At

$\lambda=1$ the ratios are benign, which is the setting used in §4.2.

Checklist. (i) Report per-partition *and* per-horizon; never aggregate across horizons without inspecting them. (ii) Judge with scale-aware prediction metrics (nMSE, decoded-pixel PSNR); use cosine and probe ρ as diagnostics only. (iii) Audit nMSE denominators for degenerate targets. (iv) Calibrate transfer against a random-init control, and validate probe protocols on a random encoder.